

Objective-C / Swift for iOS. In other cases it is about the restrictions of the target environment, for instance embedded systems programming is often done in C and Assembly. For web development, there is little stopping you from using C / C++, but that's usually a bad idea, you'd be better off with Python, Ruby, JavaScript or even PHP and Java. For scientific computing, you'll probably want MATLAB, R, or Python; here MATLAB and R were designed for this purpose, but Python is useful because of the plethora of scientific computing packages available for it. In Facebook's case they started with PHP, and once they were in too far, rather than recode everything, they decided to improve the PHP ecosystem with a PHP compiler, and now a new language called Hack based on PHP.

In this book we have featured both old and entrenched languages such as C, C++ and Java, and also newcomers such as Rust and Swift. We have languages covering most of the popular paradigms listed above. For each language we will explore how it came to be, its strengths and significance, popular and common tools for the language, and we mention a few places you can go to learn more about them. For each language we also include small code samples, one of a regular 'Hello World' program, and another of a simple insertion sort algorithm where ever possible, so you can have a richer taste of the language.

So with that in mind, what do we expect you to do with all the programming languages covered in this text? Explore them all of course! It is important to not tie oneself to a single paradigm, functional or imperative, static or dynamic, interpreted or compiled. They all have strengths and weaknesses that make them shine in their own domain. Fun fact, there is no such thing as an interpreted language or a compiled language; whether a language is interpreted or compiled is dependent on the nature of its implementation. It is quite possible to compile Python or Ruby, and also quite possible to interpret C / C++

Having a preference is fine, but being flexible is more important. No matter your feelings about a given language, it might have strengths that make it most suitable for the software you are developing, even if that strength is merely its popularity. On that note, happy coding! 